МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №6

з дисципліни «Алгоритми та структури даних»

Виконав:

студент групи ІМ-о51

Машута О.І

Перевірив:

ас. Череватенко О.В.

Київ 2026

Постановка задачі 1. Представити напрямлений граф із заданими параметрами так само, як у лабораторній роботі №3. Відмінність:

коефіцієнт k 1.0 n3 0.01 n4 0.005 0.15. Отже, матриця суміжності Adir напрямленого графа за варіантом фор мується таким чином: 1) встановлюється параметр (seed) генератора випадкових чисел, рів не номеру варіанту n1n2n3n4; 2) матриця розміром n n заповнюється згенерованими випадковими числами в діапазоні [0, 2.0); 3) обчислюється коефіцієнт k 1.0 n3 0.01 n4 0.005 0.15, кожен елемент матриці множиться на коефіцієнт k; 4) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0. 2. Створити програму, яка виконує обхід напрямленого графа вшир (BFS) та вглиб (DFS). • обхід починати з вершини із найменшим номером, яка має щонай менше одну вихідну дугу; • при обході враховувати порядок нумерації; 46 • у програмі виконання обходу відображати покроково, черговий крок виконувати за натисканням кнопки у вікні або на клавіатурі. 3. Під час обходу графа побудувати дерево обходу. У програмі дерево об ходу виводити покроково у процесі виконання обходу графа. Це можна виконати одним із двох способів: • або виділяти іншим кольором ребра графа; • або будувати дерево обходу поряд із графом. 4. Зміну статусів вершин у процесі обходу продемонструвати зміною ко льорів вершин, графічними позначками тощо, або ж у процесі обходу виводити протокол обходу у графічне вікно або в консоль. 5. Якщо після обходу графа лишилися невідвідані вершини, продовжува ти обхід з невідвіданої вершини з найменшим номером, яка має щонай менше одну вихідну дугу. При проєктуванні програми також слід врахувати наступне: 1) мова програмування обирається студентом самостійно; 2) графічне зображення усіх графів має формуватися програмою з тими жвимогами, як у ЛР №3; 3) всі графи обов’язково зображувати у графічному вікні; 4) типи та структури даних для внутрішнього представлення всіх даних у програмі слід вибрати самостійно

Текст програми using System; using System.Collections.Generic; using System.Drawing; using System.Drawing.Drawing2D; using System.Linq; using System.Windows.Forms;

namespace GraphLab6 {

```
static class Program
{
    [STAThread]
```

```
    static void Main()
    {
Application.SetHighDpiMode(HighDpiMode.SystemAware);
        Application.EnableVisualStyles();
Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new Form1());
    }
}

public class Edge : IComparable<Edge>
{
    public int U { get; set; }
    public int V { get; set; }
    public int Weight { get; set; }
    public bool IsInMST { get; set; } = false;
    public bool IsConsidered { get; set; } = false;

    public int CompareTo(Edge other)
    {
        return this.Weight.CompareTo(other.Weight);
    }
}

public class DSU
{
    private int[] parent;

    public DSU(int n)
    {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int Find(int i)
    {
```

```
        if (parent[i] == i)
            return i;
        return parent[i] = Find(parent[i]);
    }

    public void Union(int i, int j)
    {
        int rootI = Find(i);
        int rootJ = Find(j);
        if (rootI != rootJ)
        {
            parent[rootI] = rootJ;
        }
    }
}

public partial class Form1 : Form
{
    const int n1 = 3;
    const int n2 = 4;
    const int n3 = 1;
    const int n4 = 2;

    private int N;
    private List<Edge> allEdges = new List<Edge>();
    private List<Edge> sortedEdges = new List<Edge>();
    private DSU dsu;

    private int currentEdgeIndex = 0;
    private int mstWeight = 0;
    private bool algorithmStarted = false;
    private bool algorithmFinished = false;

    private PictureBox pbGraph;
    private RichTextBox txtLog;
    private Button btnGen, btnStep;
    private Label lblStatus;
```

```
    public Form1()
    {
        SetupUI();
        N = 10 + n3;
    }

    private void GenerateGraph(object sender, EventArgs e)
    {
        allEdges.Clear();
        sortedEdges.Clear();
        txtLog.Clear();
        mstWeight = 0;
        currentEdgeIndex = 0;
        algorithmStarted = false;
        algorithmFinished = false;
        btnStep.Enabled = true;
        btnStep.Text = "Start Kruskal's Algorithm";
        lblStatus.Text = "Graph generated.";

        int seed = int.Parse($"{n1}{n2}{n3}{n4}");
        Random rand = new Random(seed);

        double k = 1.0 - (n3 * 0.01) - (n4 * 0.005) -
0.05;
        txtLog.AppendText($"Variant: {n1}{n2}{n3}{n4}.
Method: Kruskal (n4={n4} is even).\n");
        txtLog.AppendText($"Coefficient K = {k:F3}\n");

        int[,] adj = new int[N, N];
        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                double val = rand.NextDouble() * 2.0;
                if (val * k >= 1.0) adj[i, j] = 1; else
adj[i, j] = 0;
```

```
            }
        }
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                if (adj[i, j] == 1 || adj[j, i] == 1)
                    adj[i, j] = adj[j, i] = 1;

        Random randW = new Random(seed);

        txtLog.AppendText("\nEdge list (u - v :
weight):\n");

        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                double b_val = randW.NextDouble() * 2.0;

                if (i < j && adj[i, j] == 1)
                {
                    int weight = (int)Math.Ceiling(b_val *
100 * 1);
                    if (weight == 0) weight = 1;

                    Edge newEdge = new Edge { U = i, V =
j, Weight = weight };
                    allEdges.Add(newEdge);
                    txtLog.AppendText($"{i + 1} - {j + 1}
: {weight}\n");
                }
            }
        }

        pbGraph.Invalidate();
    }
```

```
    private void StepKruskal(object sender, EventArgs e)
    {
        if (allEdges.Count == 0)
        {
            MessageBox.Show("Please generate the graph
first!");
            return;
        }

        if (!algorithmStarted)
        {
            sortedEdges = allEdges.OrderBy(edge =>
edge.Weight).ToList();
            dsu = new DSU(N);

            txtLog.AppendText("\n--- Start of Kruskal's
Algorithm ---\n");
            txtLog.AppendText("Edges sorted by
weight.\n");

            algorithmStarted = true;
            btnStep.Text = "Next step";
            currentEdgeIndex = 0;
            pbGraph.Invalidate();
            return;
        }

        if (algorithmFinished) return;


        if (currentEdgeIndex < sortedEdges.Count)
        {
            foreach (var ed in sortedEdges) if
(!ed.IsInMST) ed.IsConsidered = false;

            Edge eCurr = sortedEdges[currentEdgeIndex];
            eCurr.IsConsidered = true;
```

```
            int rootU = dsu.Find(eCurr.U);
            int rootV = dsu.Find(eCurr.V);

            if (rootU != rootV)
            {
                dsu.Union(rootU, rootV);
                eCurr.IsInMST = true;
                mstWeight += eCurr.Weight;
                txtLog.AppendText($"Adding: {eCurr.U + 1}
- {eCurr.V + 1} (Weight {eCurr.Weight})\n");
            }
            else
            {
                txtLog.AppendText($"Skipping: {eCurr.U +
1} - {eCurr.V + 1} (Cycle)\n");
            }

            currentEdgeIndex++;
            lblStatus.Text = $"Edges checked:
{currentEdgeIndex}/{sortedEdges.Count}. MST weight:
{mstWeight}";

            int edgesInMST = sortedEdges.Count(x =>
x.IsInMST);
            if (edgesInMST == N - 1)
            {
                txtLog.AppendText($"\nSpanning tree found!
Total weight: {mstWeight}\n");
                algorithmFinished = true;
                btnStep.Enabled = false;
                btnStep.Text = "Done";
                eCurr.IsConsidered = false;
            }
        }
        else
        {
```

```
            algorithmFinished = true;
            btnStep.Enabled = false;
            btnStep.Text = "Done";
            MessageBox.Show($"Algorithm completed.
Spanning tree weight: {mstWeight}");
        }

        txtLog.ScrollToCaret();
        pbGraph.Invalidate();
    }

    private void pbGraph_Paint(object sender,
PaintEventArgs e)
    {
        e.Graphics.SmoothingMode =
SmoothingMode.AntiAlias;
        e.Graphics.TextRenderingHint =
System.Drawing.Text.TextRenderingHint.AntiAlias;

        if (allEdges.Count == 0 && !algorithmStarted)
return;

        int w = pbGraph.Width;
        int h = pbGraph.Height;
        int r = Math.Min(w, h) / 2 - 50;
        int cx = w / 2;
        int cy = h / 2;

        PointF[] nodes = GetCircleCoords(N, cx, cy, r);

        foreach (var edge in allEdges)
        {
            if (edge.IsInMST) continue;

            Pen p = Pens.LightGray;
            if (edge.IsConsidered) p = new
Pen(Color.Orange, 2);
```

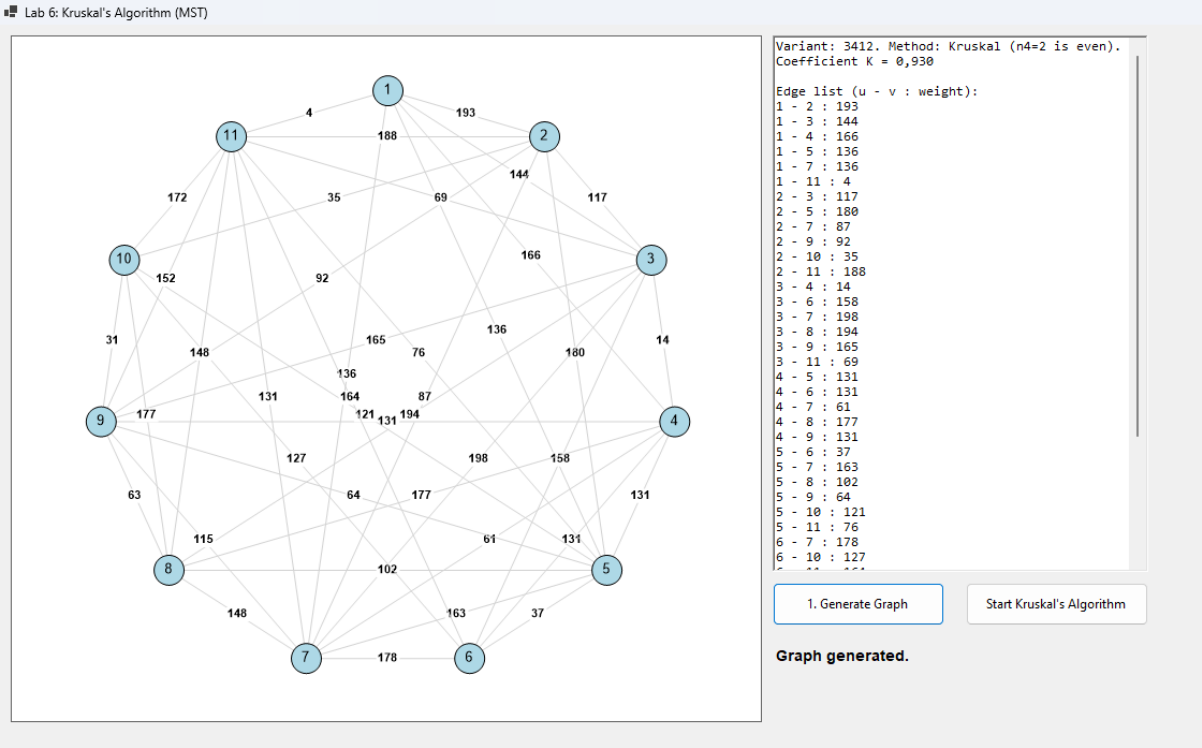
```
            DrawEdge(e.Graphics, p, nodes[edge.U],
nodes[edge.V], edge.Weight);
        }

        Pen mstPen = new Pen(Color.Red, 3);
        foreach (var edge in allEdges)
        {
            if (edge.IsInMST)
            {
                DrawEdge(e.Graphics, mstPen,
nodes[edge.U], nodes[edge.V], edge.Weight);
            }
        }

        for (int i = 0; i < N; i++)
        {
            FillNode(e.Graphics, nodes[i], (i +
1).ToString());
        }
    }

    private void DrawEdge(Graphics g, Pen p, PointF p1,
PointF p2, int weight)
    {
        g.DrawLine(p, p1, p2);

        float midX = (p1.X + p2.X) / 2;
        float midY = (p1.Y + p2.Y) / 2;

        string txt = weight.ToString();
        Font f = new Font("Arial", 8, FontStyle.Bold);
        SizeF size = g.MeasureString(txt, f);
        RectangleF rect = new RectangleF(midX - size.Width
/ 2, midY - size.Height / 2, size.Width, size.Height);

        g.FillRectangle(Brushes.White, rect);
```

```
        g.DrawString(txt, f, Brushes.Black, rect);
    }

    private void FillNode(Graphics g, PointF p, string
label)
    {
        float r = 14;
        RectangleF rect = new RectangleF(p.X - r, p.Y - r,
2 * r, 2 * r);
        g.FillEllipse(Brushes.LightBlue, rect);
        g.DrawEllipse(Pens.Black, rect);

        StringFormat sf = new StringFormat() { Alignment =
StringAlignment.Center, LineAlignment =
StringAlignment.Center };
        g.DrawString(label, new Font("Arial", 10),
Brushes.Black, rect, sf);
    }

    private PointF[] GetCircleCoords(int count, int cx,
int cy, int r)
    {
        PointF[] pts = new PointF[count];
        double angleStep = 2 * Math.PI / count;
        for (int i = 0; i < count; i++)
        {
            pts[i] = new PointF(
                (float)(cx + r * Math.Cos(i * angleStep -
Math.PI / 2)),
                (float)(cy + r * Math.Sin(i * angleStep -
Math.PI / 2))
            );
        }
        return pts;
    }

    private void SetupUI()
```

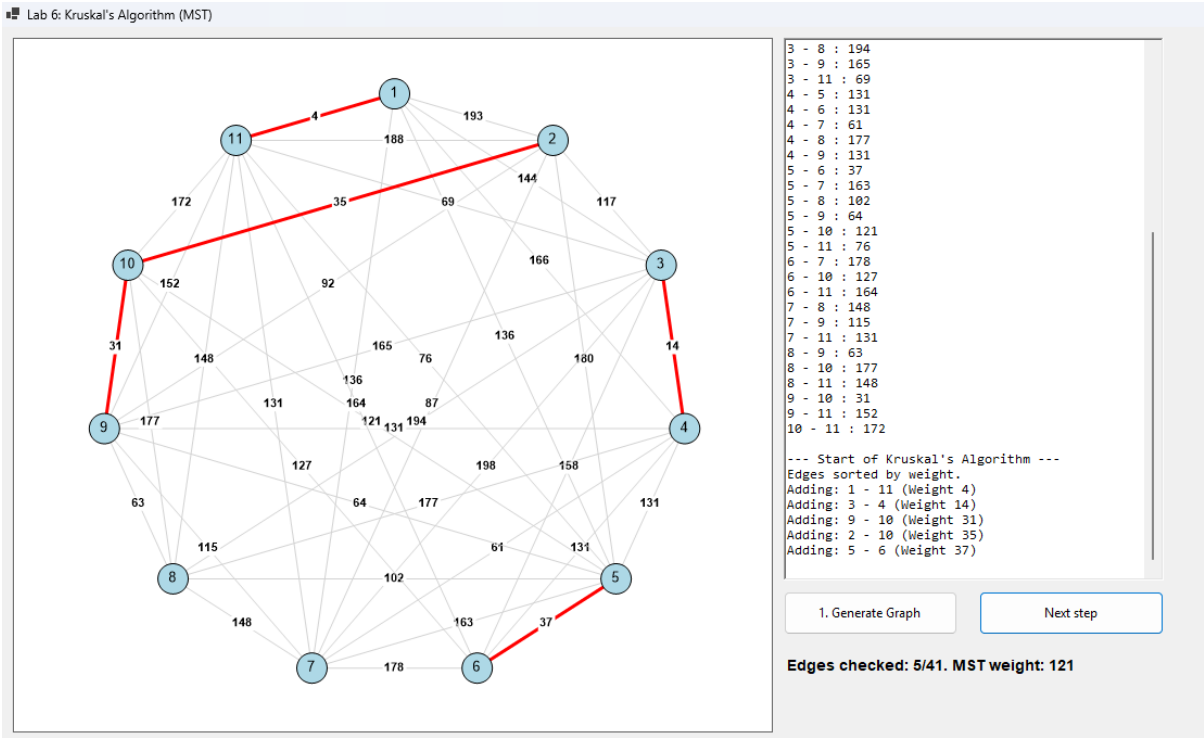
```
    {
        this.Size = new Size(1100, 700);
        this.Text = "Lab 6: Kruskal's Algorithm (MST)";

        pbGraph = new PictureBox();
        pbGraph.Location = new Point(10, 10);
        pbGraph.Size = new Size(700, 640);
        pbGraph.BorderStyle = BorderStyle.FixedSingle;
        pbGraph.BackColor = Color.White;
        pbGraph.Paint += pbGraph_Paint;
        this.Controls.Add(pbGraph);

        txtLog = new RichTextBox();
        txtLog.Location = new Point(720, 10);
        txtLog.Size = new Size(350, 500);
        txtLog.Font = new Font("Consolas", 9);
        this.Controls.Add(txtLog);

        btnGen = new Button() { Text = "1. Generate
Graph", Location = new Point(720, 520), Size = new
Size(160, 40) };
        btnGen.Click += GenerateGraph;
        this.Controls.Add(btnGen);

        btnStep = new Button() { Text = "2. Start
Algorithm", Location = new Point(900, 520), Size = new
Size(170, 40) };
        btnStep.Click += StepKruskal;
        this.Controls.Add(btnStep);

        lblStatus = new Label() { Location = new
Point(720, 580), AutoSize = true, Font = new Font("Arial",
10, FontStyle.Bold), Text = "Waiting..." };
        this.Controls.Add(lblStatus);
    }
}
```

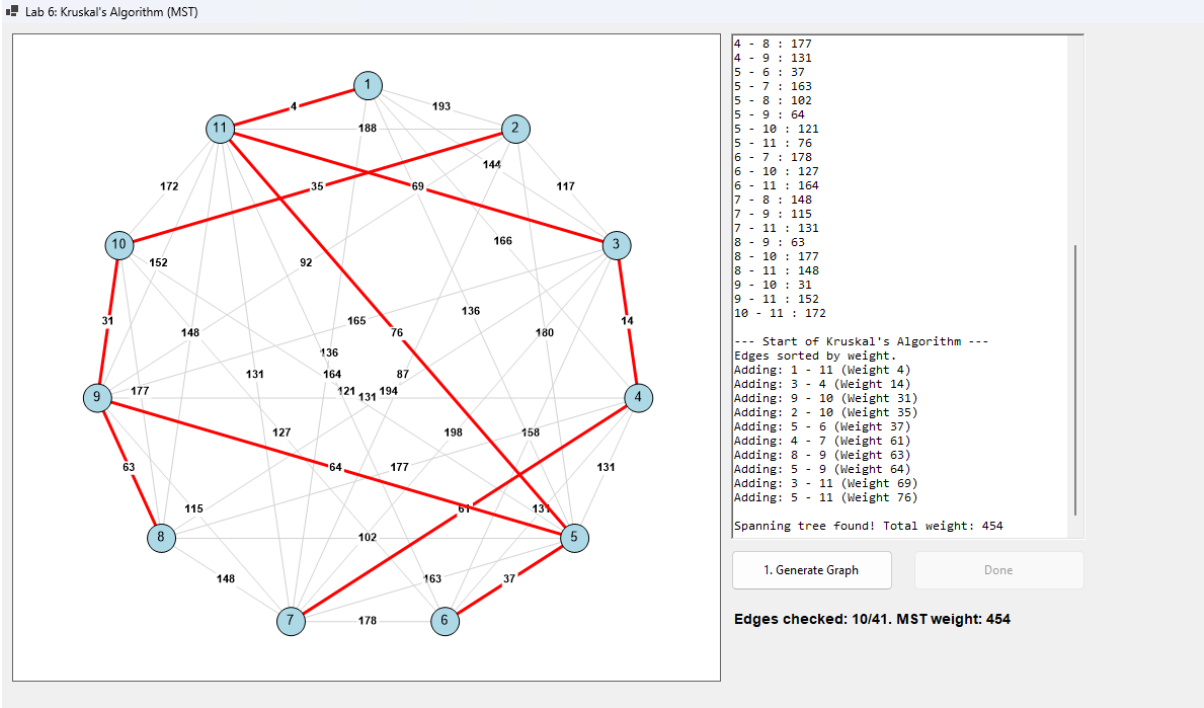
}

Тест прогами:




```
5 - 10 : 121
5 - 11 : 76
6 - 7 : 178
6 - 10 : 127
6 - 11 : 164
7 - 8 : 148
7 - 9 : 115
7 - 11 : 131
8 - 9 : 63
8 - 10 : 177
8 - 11 : 148
9 - 10 : 31
9 - 11 : 152
10 - 11 : 172
```





Висновок: У ході виконання лабораторної роботи №6 було вивчено та програмно реалізовано методи пошуку мінімального остовного (кістякового) дерева графа. Було застосовано алгоритм Краскала , який дозволив ефективно визначити підмножину ребер, що з’єднують усі вершини графа без утворення циклів з мінімальною загальною вагою. У процесі роботи було успішно виконано генерацію матриць суміжності та ваг згідно з індивідуальним варіантом завдання. Покрокова візуалізація алгоритму, включно з виділенням ребер остова та динамічним відстеженням циклів за допомогою структури даних DSU, дозволила наочно продемонструвати логіку функціонування методу. Результати розрахунків ваг

ребер та сумарної ваги остова підтверджують коректність розробленого програмного забезпечення.